ASSIGNMENT BACKEND ENGINEER

Greenhouse IoT Backend

Sensor ingestion, kontrol perangkat via MQTT,
dan health check dalam satu alur event-driven

FastAPI | MySQL | MQTT (Mosquitto) | Docker Compose

PT. DMC Indonesia Food

RINGKASAN

Tiga endpoint wajib, satu alur data IoT yang utuh

POST

`/sensor-data`

INGESTION

Validasi strict (tipe, rentang, timestamp), simpan ke MySQL, balas 201 atau error yang jelas.

POST

`/device-control`

KONTROL PERANGKAT

Validasi device_id dan ON/OFF, lalu publish ke greenhouse/control/{device_id} dengan QoS 1.

GET

`/status`

HEALTH CHECK

Status backend, database, dan MQTT. 200 jika sehat, 503 jika ada dependency yang mati.

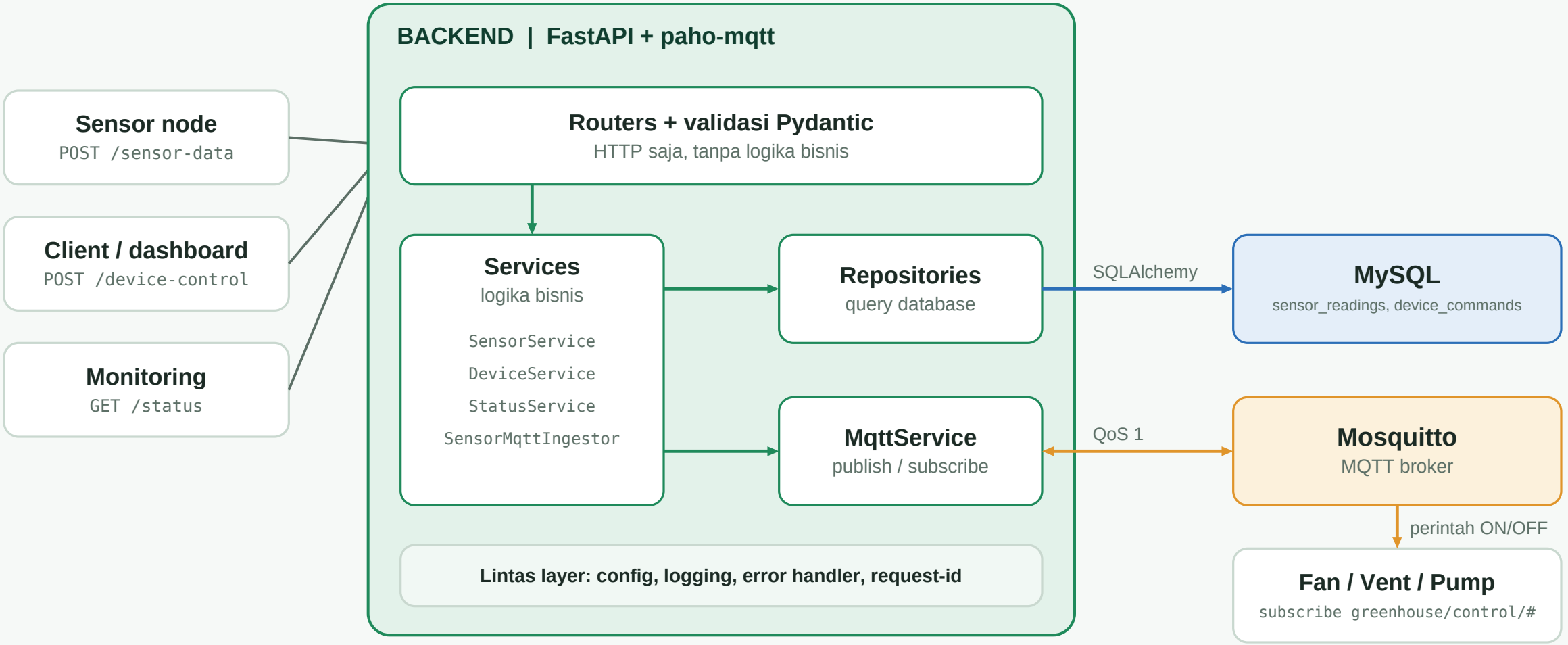
TAMBAHAN DI LUAR SYARAT

Ingestion via MQTT | **Audit log perintah** | **API key opsional** | **Swagger UI (/docs)** | **Request-ID** | **66 automated test**

Fokus penilaian: struktur kode, konsistensi API, penggunaan MQTT yang benar, pemahaman perilaku real-time, dan kualitas dokumentasi.

ARSITEKTUR

Layered architecture, MQTT sebagai jalur event

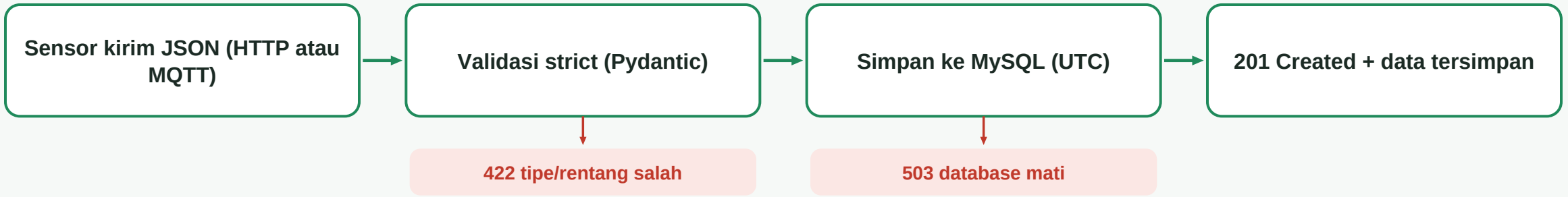


Sensor juga dapat publish ke `greenhouse/sensor/{id}`; backend subscribe dan memakai **service yang sama** dengan jalur HTTP.

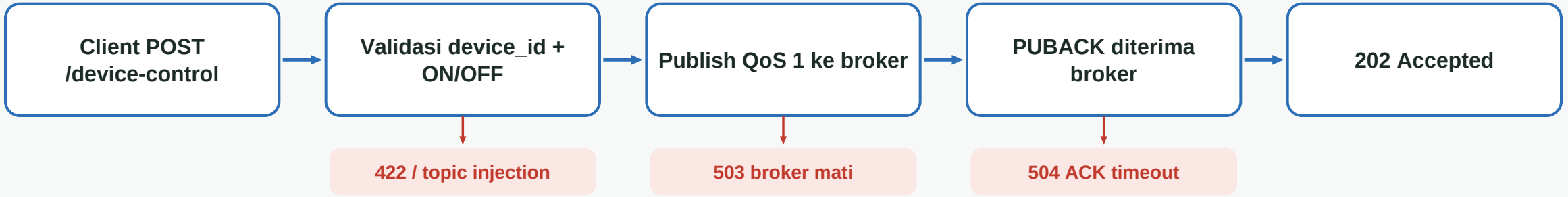
ALUR DATA

Dua alur utama: ingestion dan kontrol

ALUR A: SENSOR KE DATABASE



ALUR B: CLIENT KE PERANGKAT



Setelah 202, broker meneruskan pesan ke perangkat. Eksekusi di perangkat berjalan **asinkron**, karena itu responsnya 202 dan bukan 200.

STRUKTUR KODE

Setiap layer punya satu tanggung jawab

Layer	Tanggung jawab	Contoh file
Router	HTTP saja: terima request, panggil service, bentuk response	routers/device.py
Schema	Kontrak data dan validasi (Pydantic)	schemas/sensor.py
Service	Logika bisnis: bentuk topic, publish, audit, aturan ingestion	services/device_service.py
Repository	Query database, tanpa aturan bisnis	repositories/sensor_repository.py
Model	Definisi tabel SQLAlchemy	models/sensor_reading.py
Core	Config, exception, keamanan, logging	core/, config.py

STACK

- FastAPI
- Pydantic v2
- SQLAlchemy 2
- PyMySQL
- paho-mqtt 2
- Mosquitto 2
- pytest
- Docker Compose

Satu service, dua pintu masuk. `SensorService` dipakai oleh HTTP dan oleh subscriber MQTT, sehingga validasi dan penyimpanan identik.

create_app(settings, mqtt) menerima dependency dari luar, jadi test berjalan tanpa broker atau MySQL asli.

DESAIN API

Kontrak yang konsisten di semua endpoint

Endpoint	Sukses	Error	Catatan
`POST /sensor-data`	201 Created	400, 422, 503	Validasi strict, timestamp disimpan UTC
`POST /device-control`	202 Accepted	401, 422, 503, 504	Publish QoS 1, API key opsional
`GET /status`	200 OK	503 (degraded)	Cek DB (SELECT 1) dan koneksi MQTT
`GET /sensor-data`	200 OK	422	Verifikasi data tersimpan (tambahan)

RESPONSE SUKSES

```
{
  "success": true,
  "message": "Sensor data stored",
  "data": { "id": 1, "sensor_id": "sensor-01",
            "temperature": 28.5, "humidity": 72.3,
            "timestamp": "2026-09-21T10:00:00Z" }
}
```

RESPONSE ERROR

```
{
  "success": false,
  "error": { "code": "VALIDATION_ERROR",
    "message": "Request validation failed",
    "details": [{ "field": "humidity",
       "message": "Input should be <= 100" }],
    "request_id": "dbb0b5ab..." }
}
```

INTEGRASI MQTT

Topic terstruktur, QoS 1, reconnect otomatis

Topic	Arah	Isi
`greenhouse/control/{device_id}`	Backend ke perangkat	Perintah ON / OFF
`greenhouse/sensor/{sensor_id}`	Sensor ke backend	Data ukur (opsional)
`greenhouse/backend/status`	Backend ke semua	online / offline (retained + LWT)

PAYLOAD PERINTAH (JSON, bukan string polos)

```
{"command_id": "90d84241-9233-402d-9c0a-066912fcf48e",
 "device_id": "fan-01", "command": "ON",
 "issued_at": "2026-09-21T10:11:34.971277Z"}
```

command_id sama dengan yang tersimpan di tabel audit dan yang dikembalikan ke client, sehingga satu perintah dapat ditelusuri dari API sampai ke perangkat.

Topic injection dicegah. device_id hanya boleh berisi huruf, angka, '_' dan '-' (maks. 64). Karakter / # + ditolak dengan 422, sehingga satu request tidak bisa menarget banyak perangkat atau keluar dari namespace topic.

QoS 1

Perintah tidak boleh hilang. Backend menunggu PUBACK sebelum menjawab sukses.

Retain

Broker menyimpan perintah terakhir; perangkat yang baru online langsung menerimanya.

Auto-reconnect

Backoff 1 sampai 30 detik; semua subscription didaftarkan ulang di on_connect.

Non-blocking

Client jalan di thread jaringan paho; event loop FastAPI tidak tertahan.

Last Will

Backend mati mendadak: broker mengumumkan offline. Shutdown normal juga mengirim offline.

PERILAKU REAL-TIME

Gagal cepat, pulih sendiri

1. Normal

/status 200

/device-control 202

→

2. Broker mati

/status 503 (mqtt: disconnected)

/device-control 503 langsung

→

3. Broker hidup

paho reconnect (backoff)

subscription didaftarkan ulang

→

4. Pulih

/status 200

ingestion MQTT jalan lagi

Fail fast, tanpa antrean offline

Perintah yang tertahan lalu terkirim sejam kemudian bisa membahayakan tanaman. Client langsung tahu gagal dan memutuskan sendiri.

202 Accepted, bukan 200

PUBACK membuktikan broker menerima pesan, bukan bahwa perangkat sudah menjalankannya. Eksekusi bersifat asinkron.

Audit best-effort

Jika database mati, publish tetap berhasil. Kegagalan mencatat audit hanya masuk log dan tidak membatalkan perintah.

Skenario 1 sampai 4 diuji dengan Mosquitto sungguhan: broker dimatikan, lalu dinyalakan kembali.

ERROR HANDLING

Edge case ditangani secara eksplisit

Situasi	Perilaku
"temperature": "28.5" (string), true, NaN	422: validasi strict, tanpa koersi diam-diam
humidity 101, temperature 150	422: di luar rentang wajar sensor
timestamp di masa depan (> 5 menit)	422; zona waktu lain dikonversi ke UTC
command "on" atau " Off "	Dinormalisasi menjadi ON / OFF
command "TOGGLE"	422: hanya ON atau OFF
device_id "#", "a/b", "+"	422: whitelist karakter, mencegah topic injection
Broker MQTT mati	503 MQTT_UNAVAILABLE; audit tercatat FAILED
Database mati	503 DATABASE_UNAVAILABLE; aplikasi tidak crash
Exception tak terduga	500 generik + request_id; detail hanya di log

PRINSIP

- Satu format error untuk semua endpoint
- code stabil untuk mesin, message untuk manusia
- 4xx salah client, 5xx masalah server
- Tidak membocorkan detail internal
- X-Request-ID untuk korelasi dengan log

VERIFIKASI

Diuji otomatis dan end-to-end dengan komponen asli

66

automated tests lulus

tanpa broker atau MySQL asli
(SQLite sementara + MQTT tiruan)

Cakupan test
Validasi payload, tipe dan rentang, topic injection, normalisasi command, broker mati (503), timeout (504), database mati (503), pemulihan tanpa restart, API key, dan ingestion via MQTT.

DIVERIFIKASI DENGAN KOMPONEN ASLI

- ✓ **Mosquitto 2 sungguhan**
  Publish sampai ke subscriber; retained message diterima subscriber yang datang belakangan; status online/offline.
- ✓ **MySQL-compatible (MariaDB 10.11)**
  Tabel terbentuk otomatis; +07:00 tersimpan sebagai UTC; ingestion HTTP dan MQTT masuk ke tabel yang sama.
- ✓ **Broker dimatikan lalu dinyalakan**
  503 saat mati, reconnect otomatis dalam hitungan detik, subscription aktif lagi tanpa restart.
- ✓ **Database dimatikan**
  API menjawab 503, /status melaporkan disconnected, proses tetap hidup.
- ✓ **Graceful shutdown (SIGTERM)**
  Status retained berubah dari online menjadi offline.

DEMO

Menjalankan dan mencoba dalam tiga langkah

1 Jalankan seluruh stack

```
docker compose up --build
# MySQL + Mosquitto + backend
# Swagger: http://localhost:8000/docs
```

2 Pantau topic perintah (terminal lain)

```
mosquitto_sub -t "greenhouse/control/#" -v
```

3 Kirim perintah ke perangkat

```
curl -X POST localhost:8000/device-control \
  -H "Content-Type: application/json" \
  -d '{"device_id":"fan-01","command":"ON"}'
```

HASIL YANG DIHARAPKAN

RESPONSE API

```
HTTP 202 Accepted
"topic": "greenhouse/control/fan-01"
"status": "PUBLISHED"
```

OUTPUT MOSQUITTO_SUB (DIPOTONG)

```
greenhouse/control/fan-01 {"command_id":
 "90d84241-...", "device_id": "fan-01",
 "command": "ON", "issued_at": "..."}
```

Data sensor via MQTT: `python scripts/simulate_sensors.py`. Semua endpoint: `./scripts/demo.sh`.

PENUTUP

Trade-off dan pengembangan lanjutan

Keputusan desain dan trade-off

Fail fast vs antrean offline

Lebih aman untuk aktuator: perintah yang tertahan lalu terkirim belakangan bisa berbahaya. Tidak ada retry otomatis, client yang memutuskan.

Validasi strict + whitelist ID

Lebih banyak input ditolak (tipe salah, karakter aneh), tetapi data di database bersih dan topic MQTT aman dari injection.

HTTP dan MQTT satu service

Sedikit lebih banyak kode di awal, tetapi mencerminkan alur IoT nyata dan validasi tetap konsisten di kedua jalur.

create_all saat startup

Cukup untuk demo dan pengujian; produksi butuh migrasi berversi (Alembic) agar setiap perubahan skema tercatat, bisa di-rollback, dan aman dijalankan di banyak environment.

Pengembangan lanjutan

ACK dari perangkat

Topic greenhouse/ack/{device_id} agar status audit bisa diperbarui menjadi ACKED, bukan hanya PUBLISHED.

Keamanan broker

Mosquitto dengan password_file, ACL per perangkat, dan TLS di port 8883 untuk produksi.

Skala data sensor

Batch insert atau time-series DB (TimescaleDB / InfluxDB) jika volume data tinggi.

Rate limiting dan JWT

Saat ini baru API key statis opsional; produksi idealnya autentikasi per-user.

Beberapa instance backend

Shared subscription MQTT 5 (fitur broker) agar data sensor tidak tersimpan ganda saat backend diperbanyak.